

高性能计算课程设计

光线追踪渲染器的 并行优化

串行基准 / OpenMP 多线程 / CUDA GPU

4.02x

OpenMP 加速比

400 x 300, 64 samples, 8 threads

像素级光追

优化对象

每个像素可独立追踪光线

已编译提交

CUDA 状态

Slurm 提交 GPU 队列

课程设计任务与提交要求

- 题目自选：选择光线追踪渲染器作为可优化的现有程序
- 高性能计算技术：使用 OpenMP 和 CUDA 优化像素计算
- 可复现材料：保留原始串行代码、优化代码和运行说明
- 实验报告：覆盖背景、路线、实现、截图、结果与性能分析
- 答辩材料：用 PDF 展示问题、方法、结果和后续改进方向



源代码 + 报告 + PPT

最终提交包

统一版本，方便助教复现

为什么选择光线追踪作为优化对象

- 光线追踪能生成反射、折射、阴影和景深等真实感效果
- 每个像素需要多次采样、求交和材质散射，计算量较高
- 不同像素基本互不依赖，天然适合并行拆分
- 输出图像可直接观察，便于同时展示正确性和视觉效果



一句话概括：这是一个计算密集、并行特征明显、结果可视化强的优化题目。

光线追踪的核心计算流程

1. 相机发射光线

根据像素位置生成 primary ray

2. 场景求交

寻找最近球体交点和法线

3. 材质散射

漫反射、金属反射、玻璃折射

4. 多次采样

抗锯齿并降低随机噪声

```
for each pixel:  
    color = 0  
    for sample in samples:  
        ray = camera.get_ray(u, v)  
        color += ray_color(ray)  
    write_pixel(color / samples)
```

项目按三个版本逐步优化

V1 Serial

单线程逐像素追踪，作为性能基准



V2 OpenMP

对像素行循环并行化，利用 CPU 多核



V3 CUDA

每个 GPU thread 负责一个像素，显存保存场景数据

核心思路：保持相同渲染算法，逐步改变任务调度方式，从而对比并行化带来的性能变化

。

程序结构围绕光线、几何、材质和相机组织

vec3

三维向量运算

ray

光线起点与方向

sphere

球体求交

material

散射与反射折射

camera

视角与景深

main

场景、循环、输出

三套版本保持相似目录结构，便于比较串行逻辑、OpenMP 并行点和 CUDA kernel 改写。

串行版本提供正确性基准和性能基线

- 逐行逐像素计算颜色，逻辑最直接
- 每个像素执行 samples 次随机采样
- 递归追踪光线直到达到最大深度或返回背景色
- 输出 PPM 文件，适合在超算环境中直接生成

```
for (int j = HEIGHT - 1; j >= 0; j--) {
    for (int i = 0; i < WIDTH; i++) {
        Vec3 color(0, 0, 0);
        for (int s = 0; s < SAMPLES; s++) {
            Ray r = cam.get_ray(u, v);
            color += ray_color(r, world, materials, MAX_DEPTH);
        }
        write_pixel(out, color, SAMPLES);
    }
}
```

```
#####
# Welcome to SSH Terminal!
# IP: ymlu.fudan.edu.cn, account: bjtu
# Now login: 2020-07-01 21:46:35
#####
(base) [bjtu@login03 ~]$ cd data/serial/
(base) [bjtu@login03 ~]$ g++ -O2 -std=c++17 v1_serial.cpp -o v1_serial/v1_serial
(base) [bjtu@login03 ~]$ ./v1_serial/v1_serial --wd 10 --hgt 300 --samples 16
Rendering scanline 300/300
Done
Render time: 39.1013 s
Render time: 39.1013 s
(base) [bjtu@login03 ~]$
```

OpenMP 优化把独立像素分配给多个 CPU 线程

- 先将像素颜色写入 framebuffer，避免多线程同时写文件
- 使用 parallel for 并行图像行循环
- 使用 dynamic 调度缓解不同像素追踪深度不一致的问题
- 通过 OMP_NUM_THREADS 控制线程数，便于做扩展性分析

```
#pragma omp parallel for schedule(dynamic, 1)
for (int j = 0; j < HEIGHT; j++) {
    for (int i = 0; i < WIDTH; i++) {
        Vec3 color(0, 0, 0);
        framebuffer[row * WIDTH + i] = color;
    }
}
```

```
webssh 1 x
Done.
Render time: 39.1013 s
Render time: 39.1013 s
(base) [bjtui@login03 1]$ cat results/benchmark_results.csv
cat: results/benchmark_results.csv: No such file or directory
(base) [bjtui@login03 1]$ ls -lh results/
total 2.6M
-rw-r--r-- 1 bjtui syren_group 1.3M Jun 16 16:52 v1_output.ppm
-rw-r--r-- 1 bjtui syren_group 1.3M Jun 16 16:55 v2_output.ppm
(base) [bjtui@login03 1]$ head -n results/v1_output.ppm
head: invalid number of lines: 'results/v1_output.ppm'
(base) [bjtui@login03 1]$ head -n 5 results/v1_output.ppm
P3
400 300
255
220 235 255
220 235 255
(base) [bjtui@login03 1]$
(base) [bjtui@login03 1]$
(base) [bjtui@login03 1]$ g++ -O2 -std=c++17 -fopenmp v2_openmp/main.cpp -o v2_openmp/v2_openmp
(base) [bjtui@login03 1]$ OMP_NUM_THREADS=8 ./v2_openmp/v2_openmp --width 400 --height 300 --samples 64
Threads=8 Render time: 9.72383 s
Threads=8 Render time: 9.72383 s
(base) [bjtui@login03 1]$
```

CUDA 版本把像素计算映射到 GPU 线程

- 16 x 16 线程块组织二维网格
- 每个线程独立生成随机采样并计算一个像素颜色
- 递归 ray_color 改为迭代形式, 降低 GPU 栈压力
- 球体与材质通过 cudaMalloc 放入显存, 作为 kernel 参数传入

```
module load intel/cuda/12.1
nvcc -O3 -arch=sm_80 v3_cuda/main.cu -o
v3_cuda/v3_cuda
sbatch scripts/submit_gpu_slurm.sh
queue -u bjtu3
```

登录节点不直接跑 CUDA 程序; 正式测试通过 Slurm 提交到 gpu_4090 计算节点。

CUDA 内存设计修复了结构体动态初始化问题

- 早期版本将 Sphere / MaterialData 放入 __constant__ 数组
- 结构体中包含 Vec3 构造逻辑，导致 nvcc 报动态初始化错误
- 修复方式：在 host 端构建场景，再 cudaMemcpy + cudaMemcpyHostToDevice 到显存
- kernel 参数显式传入 spheres、materials 和 num_spheres

```
Sphere* d_spheres = nullptr;
MaterialData* d_materials = nullptr;
cudaMalloc(&d_spheres, spheres.size() * sizeof(Sphere));
cudaMalloc(&d_materials, materials.size() * sizeof(MaterialData));
cudaMemcpy(d_spheres, spheres.data(), bytes, cudaMemcpyHostToDevice);
render_kernel<<<grid, block>>>(..., d_spheres, num_spheres, d_materials);
```

可编译

结果

intel/cuda/12.1 + nvcc

更稳定

收益

避免 __constant__ 初始化限制

超算平台运行流程分为登录节点编译和 GPU 队列执行

1. 进入目录

```
cd  
data/24281100/2
```

2. 加载 CUDA

```
module load  
intel/cuda/12.1
```

3. 编译程序

```
nvcc -O3 -  
arch=sm_80 ...
```

4. 提交作业

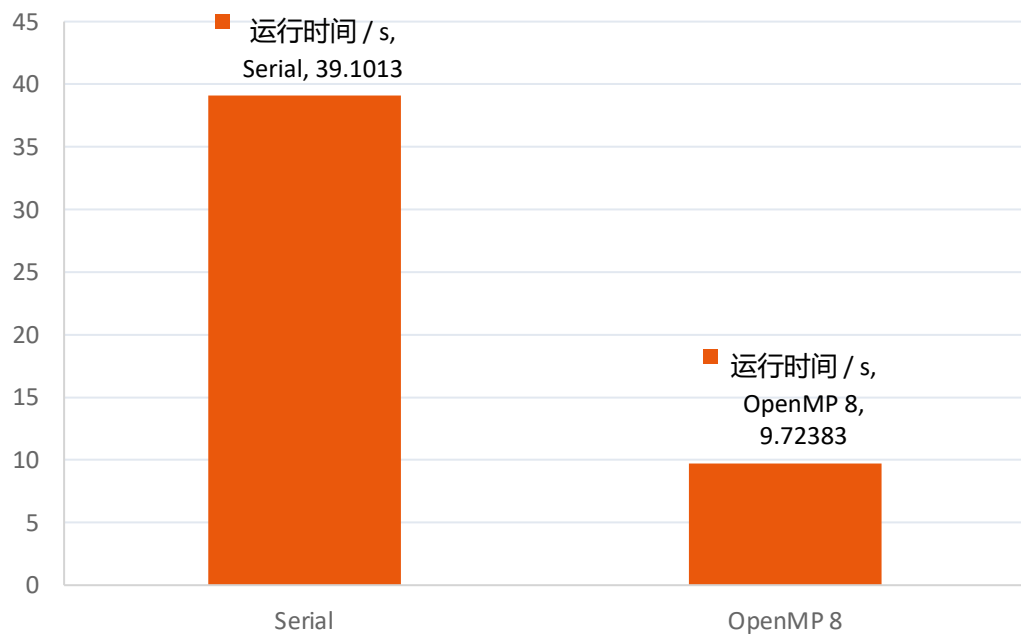
```
sbatch submit.sh
```

5. 查看队列

```
squeue -u bjtu3
```

CUDA driver version is insufficient for CUDA runtime version 出现在登录节点直接运行时；提交到 GPU 计算节点后不再作为代码错误处理。

已有超算结果显示 OpenMP 获得明显加速



39.1013 s

串行版本

400 x 300, 64 samples

9.72383 s

OpenMP 版本

8 CPU threads

加速比 = $39.1013 / 9.72383 \approx 4.02x$, 并行效率约 50.3%。

性能提升来自像素级并行，但并非理想线性加速

- 像素计算独立，因此并行化收益明显
- 文件输出、场景初始化和部分统计仍是串行部分
- 不同像素反射/折射次数不同，导致负载不均
- 随机数生成、调度和缓存访问会引入额外开销
- 线程数继续增大后，内存带宽和调度开销会限制收益

4.02x

加速比

$T_{\text{serial}} / T_{\text{openmp}}$

50.3%

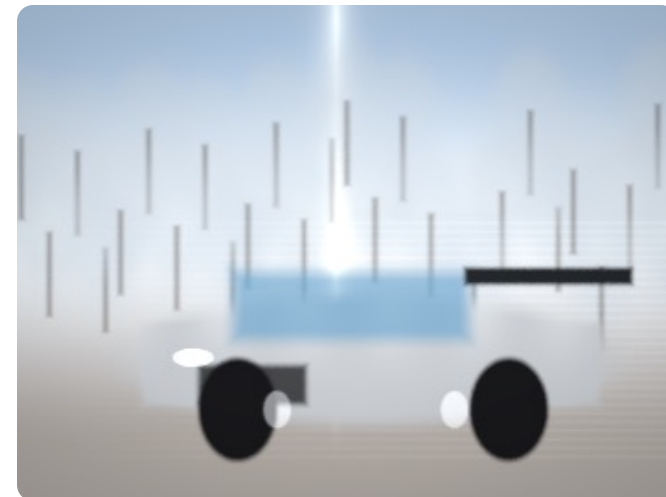
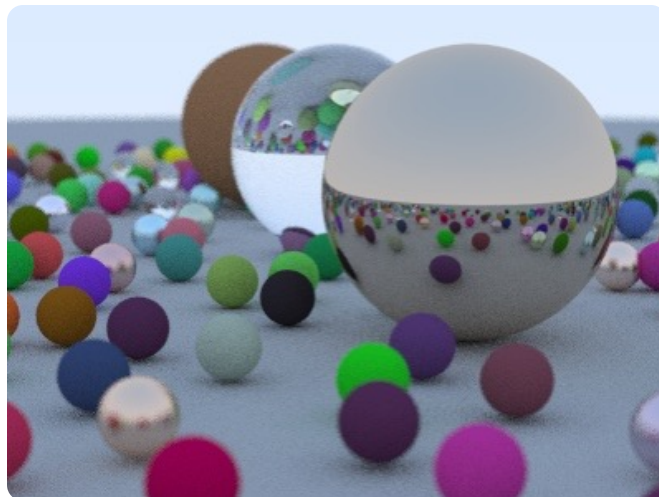
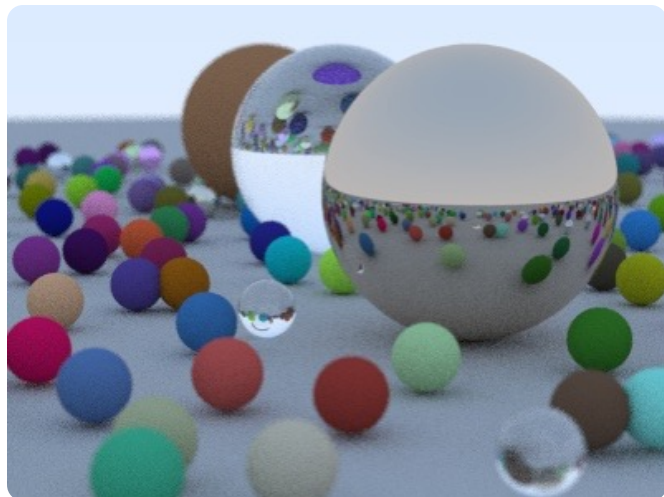
并行效率

$\text{Speedup} / 8 \text{ threads}$

结论：OpenMP 优化已经验证方向有效，CUDA 版本用于进一步挖掘 GPU 大规模并行能力。

结果图片

PPM 输出用于正确性验证，JPG/PNG 用于报告展示



PPM 适合超算环境直接输出；报告中将关键 PPM 转为 JPG，并保留 PNG 预览图。

展示目标：从基础小球走向更像游戏画面的渲染效果

参考画面强调高速运动、雪地反射、车身高光和背景运动模糊。项目中的 snow_gt / racing 场景用程序化几何近似这些视觉特征。



城市追车场景用于展示光照、反射和景深氛围

当前项目不使用商业游戏资源；画面素材作为答辩参考图，程序输出由球体、材质和像素着色过程生成。



黑洞场景用于展示程序化像素着色能力

黑洞/吸积盘展示图不依赖复杂 OBJ 模型，主要通过像素函数、噪声、环形结构和高亮边缘形成视觉效果。

复现步骤已经整理进提交包

```
g++ -O2 -std=c++17 v1_serial/main.cpp -o v1_serial/v1_serial
g++ -O2 -std=c++17 -fopenmp v2_openmp/main.cpp -o v2_openmp/v2_openmp
module load intel/cuda/12.1
nvcc -O3 -arch=sm_80 v3_cuda/main.cu -o v3_cuda/v3_cuda
OMP_NUM_THREADS=8 ./v2_openmp/v2_openmp --width 400 --height 300 --samples 64
sbatch scripts/submit_gpu_slurm.sh
```

统一版本

压缩包

组内提交内容保持一致

- SUBMISSION_README.md: 给助教的最短复现说明
- EXPERIMENT_GUIDE.md: 完整实验步骤和常见问题
- SUPERCOMPUTER_GUIDE.md: iNode、上传、module 和队列说明
- scripts/run_benchmark.sh: 统一编译和记录性能数据

目前不足与后续改进方向

- 补充 CUDA 最终运行时间，形成完整 Serial/OpenMP/CUDA 性能对比
- 加入 OBJ 模型加载，让车体从程序化球体过渡到真实三角网格
- 加入 BVH 加速结构，降低大量物体场景中的求交复杂度
- 进一步采集不同分辨率、采样数和线程数下的扩展性数据
- 将展示场景继续向实时游戏渲染风格靠近



课程设计已经形成可复现的提交包

- 完成串行、OpenMP、CUDA 三个版本的源码与运行说明
- 完成超算 CPU/OpenMP 实测，OpenMP 8 线程加速约 4.02 倍
- CUDA 已解决编译问题，并按平台要求通过 Slurm 提交 GPU 队列
- 报告中插入运行截图、结果图片、代码片段和性能分析
- 答辩 PDF 扩展为完整展示版，突出高性能计算和游戏渲染方向

谢谢老师和同学！

代码 / 报告 /

最终材料

满足课程设计提交要求